InvasiveLens — Classifying Invasive vs. Native Plant Species in Portugal

Ajimoti Rofiat Mobolaji (129556)

Instituto Superior de Agronomia, ULisboa · MSc Green Data Science 2025–2026 · Practical Machine Learning

1. Introduction

Invasive plants are one of the biggest threats to native biodiversity in Portugal. Species such as *Arundo donax*, *Cortaderia selloana* and *Acacia dealbata* grow fast, crowd out native vegetation, change habitats and are costly to remove once they take hold. The earlier they are found, the cheaper and easier they are to manage.

The difficulty is identification. Several invasive species look very similar to harmless native plants and telling them apart reliably usually needs a trained botanist. That makes large-scale monitoring slow and expensive.

In this project I ask a simple question: can a deep learning model identify these plants from a single photograph, well enough to be useful as a first screening step? My aim is not to replace experts, but to build a tool that gives a fast, honest first opinion — with a confidence score — so that field teams can decide where to send those experts. The whole project runs from one notebook that downloads its own data, trains the models, evaluates them and turns predictions into management recommendations.

2. Data

I work with five species, chosen as look-alike groups that are commonly confused in the field:

Species	Status
Acacia dealbata	invasive
Arundo donax	invasive
Cortaderia selloana	invasive
Phragmites australis	native (look-alike of *Arundo*)
Ammophila arenaria	native (coastal look-alike)

The native species come from a curated set of plant photographs. The three invasive species are downloaded automatically from **GBIF** (Global Biodiversity Information Facility), restricted to observations in Portugal that come from the iNaturalist and Pl@ntNet datasets, which keeps label quality reasonable. About 300 images per species are pulled, so the dataset can be rebuilt by anyone without me sharing large image files.

In total the notebook indexed roughly 1,600 images across the five species. Because the run reported here was on a CPU, it used a balanced subset of 120 images per species (600 in total) to keep training time sensible; on a GPU the full set is used.

3. Data organization

Images are stored in one folder per species. The notebook scans these folders and builds a table of (image path, species label, source) by reading the species name from the folder, so any newly downloaded images are picked up automatically without depending on a fixed file list.

Before training, the data is prepared as follows:

- **Cleaning:** unreadable or corrupt image files are skipped safely rather than crashing the run.
- **Balancing:** on CPU the classes are capped to an equal number of images per species so no class dominates; a class-weighted loss is also used to handle any remaining imbalance.
- **Preprocessing:** every image is resized and normalised using standard ImageNet statistics.
- **Augmentation (training only):** random crops, horizontal flips and small colour changes are applied so the model sees more variety and overfits less.
- **Splitting:** I use **stratified k-fold cross-validation** — each image is tested exactly once, in a fold it was not trained on, and every fold keeps the same class balance. This gives a more honest estimate than a single train/test split.

A note on splitting: my original intention was to group folds geographically, so photos from the same location could not appear in both training and testing (which would let the model "cheat" on background cues). The GBIF images do not carry GPS coordinates, so true geographic splitting was not possible on this data, and stratified splitting is the honest alternative. The notebook switches back to geographic grouping automatically if coordinates are ever added.

4. Methods

Models. I compare three: a convolutional neural network built from scratch (the *baseline*), and two pre-trained models, **EfficientNetV2-S** and **ResNet50**, fine-tuned from ImageNet weights. The baseline shows what the harder models actually add.

Transfer learning. A few hundred images per class is too little to train a deep network from scratch without overfitting. Starting from ImageNet weights gives the model strong general visual features that transfer well to plant photos, so it learns the task from much less data.

Training. All models are trained with the Adam optimiser and a cross-entropy loss weighted by class frequency. Random seeds are fixed so the experiment can be repeated. Full settings are in the Appendix.

Evaluation. I report accuracy and macro-F1 (macro-F1 treats every species equally, which matters with limited data). I also collapse the five-class predictions into the two groups that matter for management — invasive vs. native — and report that separately. To check whether the best two models

really differ I use **McNemar's test**, which compares two models on the same test items. Finally I use **Grad-CAM** to visualise which parts of an image each prediction relied on.

From prediction to action. A small decision engine converts each prediction into a risk level using its confidence: invasive and very confident (≥90%) → CRITICAL (immediate removal); invasive and

confident (70–90%) → HIGH (containment); below 70% → MEDIUM (the model holds back and asks for field

verification instead of guessing); native → no action.

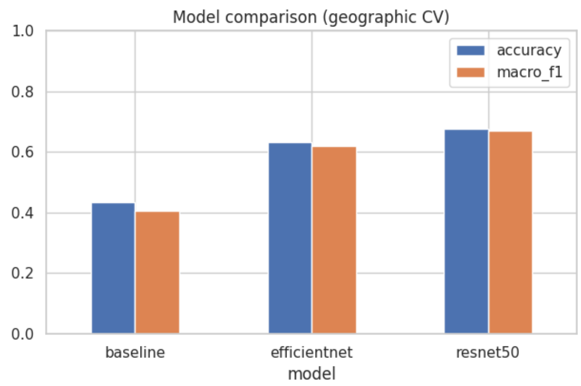
5. Results

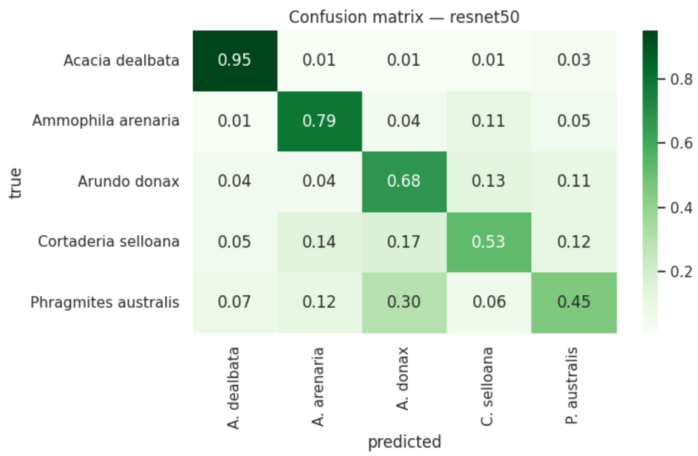
Averaged over the cross-validation folds:

Model	Accuracy	Macro-F1
Baseline CNN (from scratch)	0.43	0.41
EfficientNetV2-S	0.63	0.62
ResNet50	**0.68**	**0.67**

Per-species results for the best model (ResNet50):

Species	Precision	Recall	F1
Acacia dealbata	0.84	0.95	0.89
Ammophila arenaria	0.72	0.79	0.75
Arundo donax	0.57	0.68	0.62
Cortaderia selloana	0.63	0.53	0.57
Phragmites australis	0.60	0.45	0.51





Collapsed to the **invasive vs. native** task, ResNet50 reaches **0.79 accuracy and 0.78 macro-F1**, with the invasive class scoring **precision 0.81, recall 0.85, F1 0.83**

McNemar's test (ResNet50 vs. EfficientNet): ResNet50 was right on 92 images where EfficientNet was

wrong, versus 64 the other way, giving $\chi^2 = 4.67$, $p = 0.03$ — a statistically significant difference.

Grad-CAM heatmaps for the *Acacia* examples fall on the foliage and flowers rather than the background, suggesting the model is mostly looking at the plant. (Confusion matrix and Grad-CAM figures are in the Appendix.)

6. Analysis

The results support a few clear conclusions.

Transfer learning works. Both pre-trained models clearly beat the from-scratch baseline (0.67 macro-F1 vs. 0.41), which confirms that ImageNet features transfer well to this small dataset. ResNet50 edged out EfficientNet, and McNemar's test shows the gap is real rather than chance.

The errors make botanical sense. *Acacia dealbata* is the easiest species (0.95 recall) because its bright yellow mimosa flowers are very distinctive. The hardest cases are *Phragmites* and *Arundo*: the confusion matrix shows the model mixing these two up, and they are exactly the tall, reed-like grasses that are difficult even for people. This tells me the model is failing where humans also struggle, not in random ways.

High invasive recall is the right kind of performance. For an early-warning tool, missing an invasive plant is worse than a false alarm. The invasive class recall of 0.85 means the model rarely misses an invasive, which is the behaviour we want; the MEDIUM/abstain band then catches the uncertain cases for human review instead of forcing a wrong decision.

What is still weak. Overall accuracy is modest, partly because this was a reduced CPU run and partly because the dataset is small and the *Phragmites*/*Arundo* pair is genuinely hard. A full GPU run on all images, with more folds, should improve and stabilise the numbers.

7. Deployment

The system is delivered as a **single Colab notebook**. The user selects a GPU runtime and clicks *Run all*; the notebook downloads the native-species images (a shared Google Drive zip) and the invasive-species images (from GBIF), builds the dataset, trains and evaluates all three models, and prints every result — with no manual setup, file uploads or Drive mounting. This makes the work fully reproducible by a supervisor or a colleague.

For real use, the trained model feeds the decision engine, which outputs a risk level and a recommended action per photo, and abstains when it is not confident enough. The wider project also includes a command-line tool and a simple REST API, so the same model could be called from a field app that uploads a photo and receives a risk assessment. Before any real deployment the model would need testing on genuine field photos across seasons and conditions, and ideally expert-verified training data.

8. References

- 1. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition.* CVPR.
- 2. Tan, M., & Le, Q. (2021). *EfficientNetV2: Smaller Models and Faster Training.* ICML.
- 3. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). *ImageNet: A Large-Scale Hierarchical Image Database.* CVPR.
- 4. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* ICCV.
- 5. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* NeurIPS.
- 6. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python.* JMLR, 12, 2825–2830.
- 7. McNemar, Q. (1947). *Note on the sampling error of the difference between correlated proportions or percentages.* Psychometrika, 12(2), 153–157.
- 8. GBIF.org — Global Biodiversity Information Facility. https://www.gbif.org
- 9. iNaturalist. https://www.inaturalist.org · Pl@ntNet. https://plantnet.org
- 10. Marchante, H., Morais, M., Freitas, H., & Marchante, E. (2014). *Guia prático para a identificação de plantas invasoras em Portugal.* Imprensa da Universidade de Coimbra. (invasoras.pt)

9. Contributions

This project was carried out individually by **Ajimoti Rofiat Mobolaji**: problem definition, dataset assembly, all code (data download and organisation, model training, evaluation and the decision engine), the experiments, and this report. External resources used: training images from GBIF (iNaturalist and Pl@ntNet observations); pre-trained model weights from torchvision (ImageNet); and the open-source libraries listed in the References.

Appendix

A. Experimental setup

Setting	GPU run	CPU run (reported)
Image size	224 px	128 px
Batch size	32	16
Epochs per fold	10	3
Cross-validation folds	5	3
Images per species	all (~all available)	120
Optimiser / learning rate	Adam / 1e-4	Adam / 1e-4
Loss	class-weighted cross-entropy	class-weighted cross-entropy
Pre-trained weights	ImageNet	ImageNet
Random seed	42	42

B. Dataset counts (indexed before subsetting)

Phragmites australis 500 · *Acacia dealbata* 300 · *Arundo donax* 300 · *Cortaderia selloana* 300 · *Ammophila arenaria* 200 (≈1,600 images total).

C. Figures *(paste from the notebook output)*

- Figure 1 — Model comparison bar chart (accuracy and macro-F1).
- Figure 2 — Confusion matrix for ResNet50.
- Figure 3 — Grad-CAM overlays for sample predictions.

D. How to reproduce

Open the notebook in Google Colab → choose a GPU runtime → *Run all*. All data is downloaded automatically; no manual setup is required.